

Homework 02

Steven Jia

September 6, 2026

Question 1

(a) Simulation

I generated the design matrix one time and then kept it fixed. For every simulation I made a new training response and a new test response. I used `lm.fit` because all I need here is ordinary least squares.

```
set.seed(43202)

n <- 100
p <- 20
B <- 200

X_all <- matrix(rnorm(n * p), nrow = n, ncol = p)
beta <- 0.4^(sqrt(1:p))
mu <- as.vector(X_all %*% beta)

make_X <- function(m) {
  if (m == 0) {
    matrix(1, nrow = n, ncol = 1)
  } else {
    cbind(1, X_all[, 1:m, drop = FALSE])
  }
}

train_mse <- matrix(NA_real_, B, p + 1)
test_mse <- matrix(NA_real_, B, p + 1)
one_test_mse <- numeric(p + 1)

for (b in 1:B) {
  y <- mu + rnorm(n)
  y_test <- mu + rnorm(n)

  for (m in 0:p) {
    fit <- lm.fit(make_X(m), y)
    y_hat <- fit$fitted.values
    train_mse[b, m + 1] <- mean((y - y_hat)^2)
    test_mse[b, m + 1] <- mean((y_test - y_hat)^2)
  }

  if (b == 1) {
    one_test_mse <- test_mse[b, ]
  }
}

violations <- sum(apply(
  train_mse, 1,
```

```

function(z) any(diff(z) > 1e-10)
))

avg_train <- colMeans(train_mse)
avg_test <- colMeans(test_mse)

cat("Runs where training MSE increased:", violations, "\n")

```

```
## Runs where training MSE increased: 0
```

```

cat("Smallest average test MSE is at m =",
    which.min(avg_test) - 1, "\n")

```

```
## Smallest average test MSE is at m = 6
```

There were zero violations, so the training MSE never increased when I added a predictor. This is what should happen because the bigger model can always copy the fit from the smaller model if the new coefficient is zero.

(b) Theoretical averages

For each model I projected the true mean vector onto that model and used the leftover part to calculate B_m^2 . This avoids making the hat matrix directly.

```

m_grid <- 0:p
bias2 <- numeric(p + 1)

for (m in m_grid) {
  mu_fit <- lm.fit(make_X(m), mu)$fitted.values
  bias2[m + 1] <- sum((mu - mu_fit)^2)
}

theory_train <- bias2 / n + 1 - (m_grid + 1) / n
theory_test <- bias2 / n + 1 + (m_grid + 1) / n

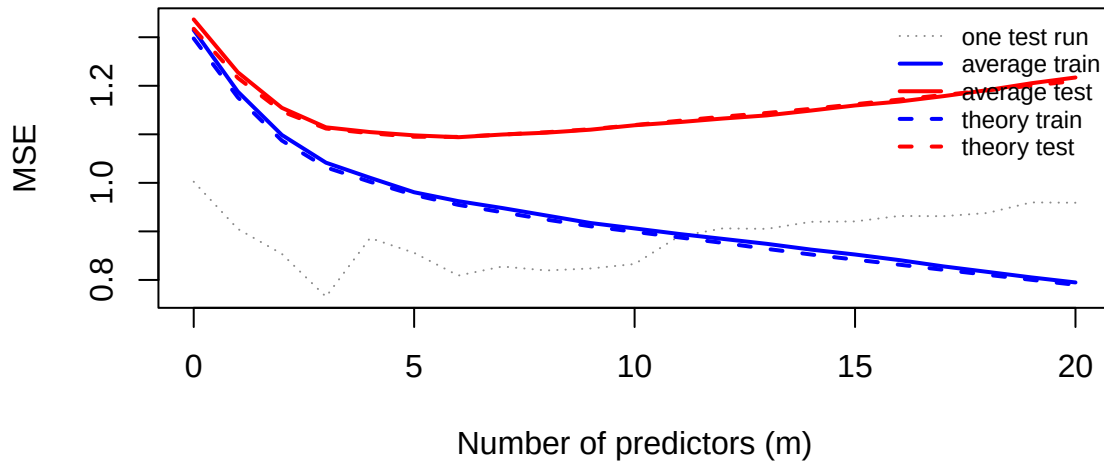
all_values <- c(
  avg_train, avg_test, theory_train,
  theory_test, one_test_mse
)

plot(
  m_grid, one_test_mse,
  type = "l", lty = 3, col = "grey55",
  xlab = "Number of predictors (m)",
  ylab = "MSE",
  ylim = range(all_values)
)

lines(m_grid, avg_train, col = "blue", lwd = 2)
lines(m_grid, avg_test, col = "red", lwd = 2)
lines(m_grid, theory_train, col = "blue", lty = 2, lwd = 2)
lines(m_grid, theory_test, col = "red", lty = 2, lwd = 2)
legend(
  "topright",
  c("one test run", "average train", "average test",
    "theory train", "theory test"),
  col = c("grey55", "blue", "red", "blue", "red"),
  lty = c(3, 1, 1, 2, 2),
  lwd = c(1, 2, 2, 2, 2),

```

```
cex = 0.72,
bty = "n"
)
```



```
cat("Largest train average/theory gap:",
    round(max(abs(avg_train - theory_train)), 4), "\n")
```

```
## Largest train average/theory gap: 0.0172
```

```
cat("Largest test average/theory gap:",
    round(max(abs(avg_test - theory_test)), 4), "\n")
```

```
## Largest test average/theory gap: 0.0193
```

```
cat("Smallest theoretical test MSE is at m =",
    which.min(theory_test) - 1, "\n")
```

```
## Smallest theoretical test MSE is at m = 6
```

The solid average curves are close to the dashed theoretical curves. The largest gaps are only about 0.017 for training and 0.019 for testing.

(c) What the curves mean

The dotted test curve is just one run, so it moves around much more. It contains the luck from one training error vector and one test error vector. Averaging 200 runs cancels a lot of that random movement.

The test formula has two competing parts. Approximation bias usually goes down when more useful predictors are included, but the estimation variance goes up with $m + 1$. In this simulation the theoretical and average test curves are both smallest at $m = 6$. Training MSE keeps going down even after this, so choosing the model only from training MSE would give too many predictors.

Question 2

(a) Theoretical error at the target point

```
n2 <- 100
p2 <- 6
sigma2 <- 1

i <- 1:n2
X2 <- outer(
  i, 1:p2,
  function(i, j) sqrt(2) * cos(pi * j * (i - 0.5) / n2)
)

beta2 <- rep(0.5, p2)
x0 <- c(0, 1, 0, 0, 1, 0)
mu2 <- as.vector(X2 %*% beta2)
mu0 <- sum(x0 * beta2)
m2 <- 0:p2

sq_bias <- sapply(m2, function(m) {
  omitted <- if (m == p2) numeric(0) else (m + 1):p2
  sum(x0[omitted] * beta2[omitted])^2
})

variance <- sapply(m2, function(m) {
  included <- if (m == 0) numeric(0) else 1:m
  sigma2 / n2 * (1 + sum(x0[included]^2))
})

theory2 <- sq_bias + variance
theory_table <- data.frame(
  m = m2,
  squared_bias = sq_bias,
  variance = variance,
  expected_error = theory2
)

cat("mu_0 =", mu0, "\n")
```

```
## mu_0 = 1
```

```
knitr::kable(theory_table, digits = 2)
```

m	squared_bias	variance	expected_error
0	1.00	0.01	1.01
1	1.00	0.01	1.01
2	0.25	0.02	0.27
3	0.25	0.02	0.27
4	0.25	0.02	0.27
5	0.00	0.03	0.03
6	0.00	0.03	0.03

Here $\mu_0 = 1$. The squared bias is the effect that is still missing because some predictors have not been included. The variance is there because the intercept and coefficients have to be estimated from noisy data.

The expected error stays the same from $m = 0$ to 1, changes at $m = 2$, stays the same through $m = 4$, changes again at $m = 5$, and stays the same at $m = 6$. These are exactly the places where predictor 2 or 5 enters.

(b) Simulation

```
set.seed(43203)

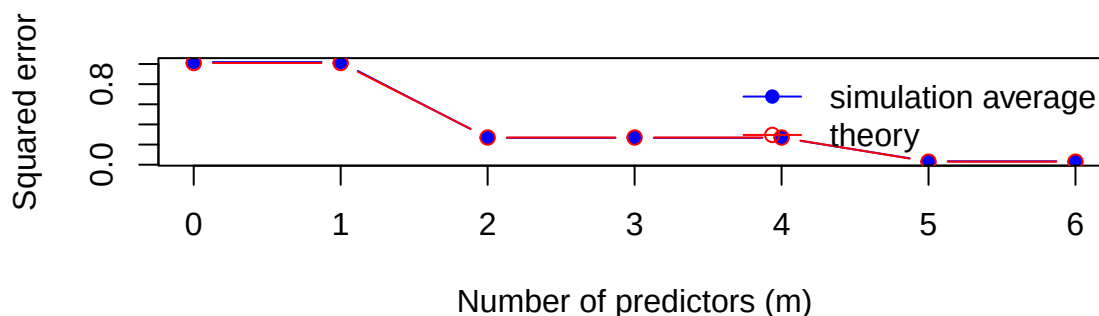
sq_error <- matrix(NA_real_, 200, p2 + 1)

for (b in 1:200) {
  y <- mu2 + rnorm(n2)

  for (m in 0:p2) {
    Xm <- cbind(1, X2[, seq_len(m), drop = FALSE])
    fit <- lm.fit(Xm, y)
    target_row <- c(1, x0[seq_len(m)])
    mu0_hat <- sum(target_row * fit$coefficients)
    sq_error[b, m + 1] <- (mu0_hat - mu0)^2
  }
}

sim_average <- colMeans(sq_error)

plot(
  m2, sim_average,
  type = "b", pch = 16, col = "blue",
  xlab = "Number of predictors (m)",
  ylab = "Squared error",
  ylim = range(c(sim_average, theory2))
)
lines(m2, theory2, type = "b", pch = 1, col = "red")
legend(
  "topright",
  c("simulation average", "theory"),
  col = c("blue", "red"),
  lty = 1,
  pch = c(16, 1),
  bty = "n"
)
```



```
cat("Smallest theoretical minimizer: m =",
    which.min(theory2) - 1, "\n")
```

```
## Smallest theoretical minimizer: m = 5
```

The simulation averages are close to the theoretical values. The smallest m that gives the minimum theoretical error is $m = 5$.

(c) Why only two predictors matter here

All six beta values are 0.5, but the target point has nonzero values only for predictors 2 and 5. In the dot product $\mathbf{x}_0^\top \beta$, every other term is zero. This is why predictors 1, 3, 4, and 6 do not directly change this target mean.

This is different from Question 1. There, test MSE averages error over all rows of the design matrix. A predictor can matter somewhere in those rows even if it does not matter at this one target point.

Question 3

(a) Training and test expectations

For training error,

$$\mathbf{y} - \mathbf{H}\mathbf{y} = (\mathbf{I} - \mathbf{H})\mu + (\mathbf{I} - \mathbf{H})\epsilon.$$

The cross term has expectation zero. Also, $(\mathbf{I} - \mathbf{H})^2 = \mathbf{I} - \mathbf{H}$, so

$$\begin{aligned} E(\|\mathbf{y} - \mathbf{H}\mathbf{y}\|^2 \mid \mathbf{X}) &= B^2 + \sigma^2 \text{tr}(\mathbf{I} - \mathbf{H}) \\ &= B^2 + \sigma^2\{n - (p + 1)\}. \end{aligned}$$

After dividing by n ,

$$E(\text{MSE}_{\text{train}} \mid \mathbf{X}) = \frac{B^2}{n} + \sigma^2 \left(1 - \frac{p + 1}{n}\right).$$

For the test response,

$$\mathbf{y}^* - \mathbf{H}\mathbf{y} = (\mathbf{I} - \mathbf{H})\mu + \epsilon^* - \mathbf{H}\epsilon.$$

The two error vectors are independent, so their cross terms have expectation zero. Their two variance contributions are $\sigma^2 n$ and $\sigma^2(p + 1)$. Therefore,

$$E(\text{MSE}_{\text{test}} \mid \mathbf{X}) = \frac{B^2}{n} + \sigma^2 \left(1 + \frac{p + 1}{n}\right).$$

(b) Expected optimism

Subtracting the training result from the test result gives

$$E(\text{MSE}_{\text{test}} - \text{MSE}_{\text{train}} \mid \mathbf{X}) = \frac{2\sigma^2(p + 1)}{n}.$$

For the numbers in the question,

$$\begin{aligned} E(\text{MSE}_{\text{train}} \mid \mathbf{X}) &= 0.04 + 1 - \frac{5}{120} = 0.9983, \\ E(\text{MSE}_{\text{test}} \mid \mathbf{X}) &= 0.04 + 1 + \frac{5}{120} = 1.0817, \\ \text{difference} &= \frac{10}{120} = 0.0833. \end{aligned}$$

These are expected values, not a rule for every single data set. Random noise can sometimes make one realized test MSE smaller than its matching training MSE.

(c) Corrected MSE and C_p

$$\widehat{\text{MSE}}_{\text{test}} = \frac{116.4 + 2(5)(0.96)}{120} = 1.05,$$

and

$$C_p = \frac{116.4}{0.96} - 120 + 2(5) = 11.25.$$

Rearranging the definition of C_p gives

$$\widehat{\text{MSE}}_{\text{test}} = \frac{\hat{\sigma}^2}{n}(C_p + n).$$

If every candidate model uses the same n and $\hat{\sigma}^2$, these are just a positive rescaling plus a constant. So the model with the smaller C_p also has the smaller corrected test MSE.

Question 4

(a) Fits and common variance estimate

```
diabetes <- read.csv("data/diabetes.csv")
train <- diabetes[1:370, ]

fit_A <- lm(
  y ~ bmi + bp + s5 + sex + s1 + s2 + s4,
  data = train
)
fit_B <- lm(
  y ~ bmi + bp + s5 + sex + s1 + s2 + s4 + s6,
  data = train
)
fit_full <- lm(y ~ ., data = train)

rss <- c(
  "Model A" = deviance(fit_A),
  "Model B" = deviance(fit_B),
  "Full reference" = deviance(fit_full)
)
p_count <- c(7, 8, 10)
full_df <- df.residual(fit_full)
sigma_hat2 <- rss["Full reference"] / full_df

fit_table <- data.frame(
  model = names(rss),
  p = p_count,
  RSS = unname(rss)
)

knitr::kable(fit_table, digits = 2)
```

model	p	RSS
Model A	7	1098909
Model B	8	1089717
Full reference	10	1089452

```
cat("Full model residual degrees of freedom:", full_df, "\n")
```

```
## Full model residual degrees of freedom: 359
```

```
cat("Common variance estimate:",  
    round(sigma_hat2, 3), "\n")
```

```
## Common variance estimate: 3034.684
```

I used only rows 1 through 370 here. The full model has 359 residual degrees of freedom, and the common variance estimate is 3034.684.

(b) Three criteria

```
n_train <- 370  
p_candidates <- c("Model A" = 7, "Model B" = 8)  
candidate_rss <- rss[c("Model A", "Model B")]  
k <- p_candidates + 1  
  
cp <- candidate_rss / sigma_hat2 - n_train + 2 * k  
aic_red <- n_train * log(candidate_rss / n_train) + 2 * k  
bic_red <- n_train * log(candidate_rss / n_train) +  
  k * log(n_train)  
  
criteria_table <- data.frame(  
  model = names(candidate_rss),  
  Cp = unname(cp),  
  reduced_AIC = unname(aic_red),  
  reduced_BIC = unname(bic_red)  
)  
  
knitr::kable(criteria_table, digits = 3)
```

model	Cp	reduced_AIC	reduced_BIC
Model A	8.116	2974.640	3005.948
Model B	7.087	2973.532	3008.754

C_p selects Model B because 7.087 is smaller than 8.116. Reduced AIC also selects Model B because 2973.532 is smaller than 2974.640. Reduced BIC selects Model A because 3005.948 is smaller than 3008.754. I only compared values inside each criterion.

(c) Why the criteria disagree

Adding s_6 lowers RSS by 9191.509. On the C_p scale this improvement is $9191.509/3034.684 = 3.029$, which is bigger than the extra penalty of 2. For reduced AIC, the improvement in the log-RSS part is 3.108, also bigger than its extra penalty of 2. This is why both pick Model B.

For BIC, the extra one-parameter penalty is $\log(370) = 5.914$. That is bigger than the 3.108 improvement, so BIC stays with Model A.

Picking a model only says which candidate did best under that score and penalty. It does not prove that the model contains the exact true predictors or that the real relationship is perfectly linear.

Question 5

(a) Misuse of the final test data

The problem starts in step 2, when all eleven candidate models are checked on the final test data for the purpose of choosing among them. Step 3 then makes the selection explicit. At that point the test set is acting like a validation set, even though it was not used to estimate coefficients.

Each test MSE has some random luck in it. Taking the smallest of eleven values tends to choose a model that had favorable luck on these particular test rows. Because of this, the minimum is usually too low as an estimate of future error for the whole selection procedure.

(b) Five-fold cross-validation

I would split the first 370 rows into five folds. For each $m = 0, \dots, 10$, I would fit the model on four folds and calculate MSE on the fold left out. After every fold has been left out once, I would average the five MSE values and choose the m with the smallest average.

Then I would refit that one chosen model using all 370 training rows. Only after the predictor count is fixed would I calculate one MSE on rows 371 through 442. That last number is the final test estimate.

(c) If the test results were already viewed

The analyst can still show all eleven test MSE values and clearly say that they were used for exploration and selection. The minimum can be reported as a test-informed result, but not as a fresh final evaluation. The cross-validation results from the training rows can also be reported separately.

For a new final evaluation, another independent set of observations from the same target population would be needed. Those observations must not be examined until the full selection and fitting procedure is fixed.